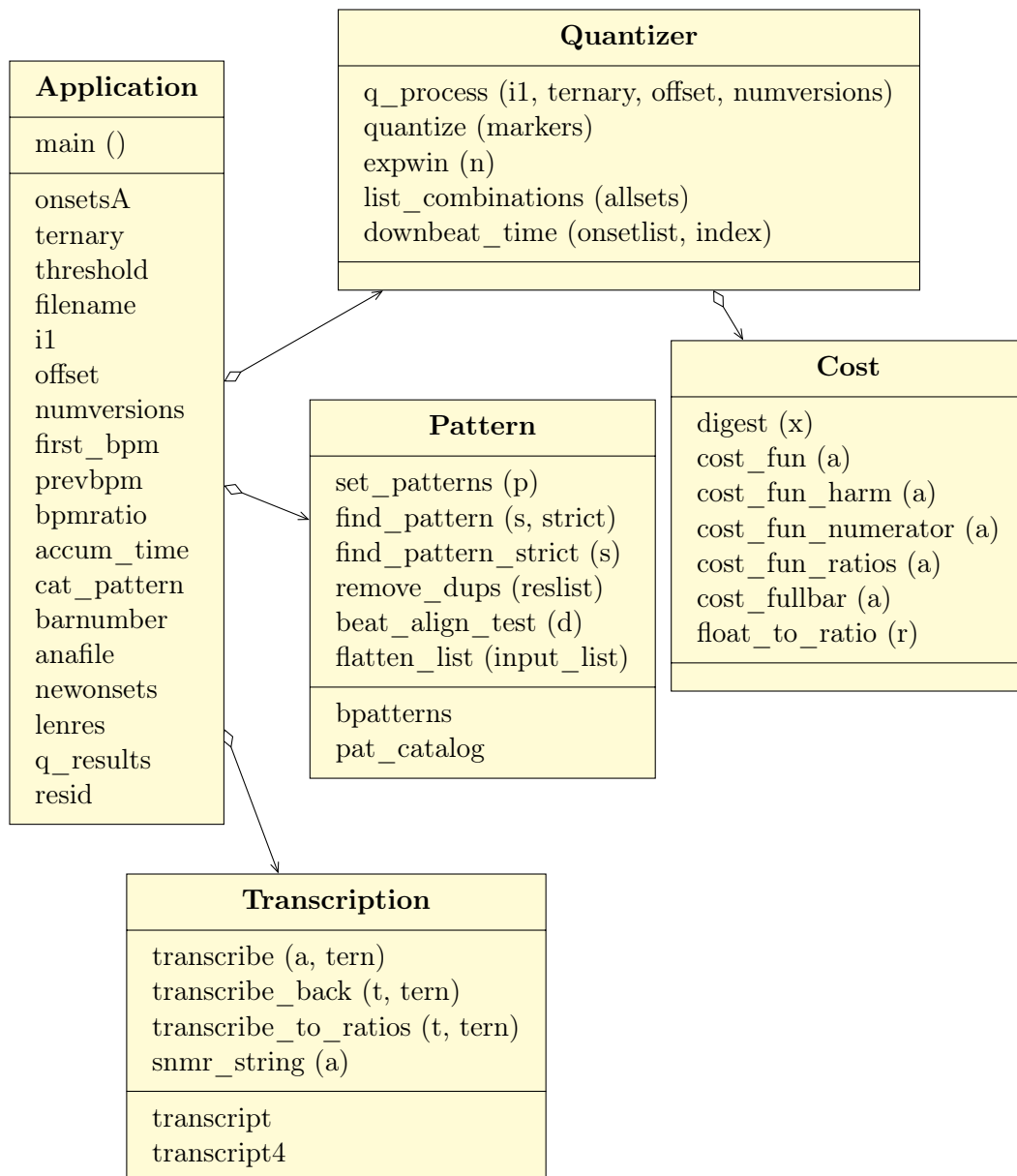


Automated Quantization and Transcription

Georg Boenn

June 5, 2020

1 Overview



2 Application class

Application
main ()
onsetsA ternary threshold filename il offset numversions first_bpm prevbpm bpmratio accum_time cat_pattern barnumber anafile newonsets lenres q_results resid

The main program collects all command-line arguments. It selects the list of patterns from the database, which is used in order to validate the outcomes of the quantization process. Main opens the file containing all note onsets measured in the musical performance. It creates two output file pointers `t_onsets` for the list of quantized onsets, as well as for an analysis file, `t_analysis`, which will contain additional information. Main creates a while loop in which the `q_process` method is repeatedly called over chunks of onsets extracted from the onset list. Finally, main collects and analyzes the results returned by `q_process` and writes them to the output files. The command-line arguments are listed in table 2.

Example invocation:

```
python3.7 grouping_dev121.py Chopin_Mazurka_41_4.txt 0.001 1 0 1 3 5 chopin
```

Table 1: Application class member variables

symbol	meaning
onsetsA	list holding the onset markers in seconds
ternary	flag to set binary or ternary meter
threshold	in seconds for a duration to fall into the same class
filename	the textfile with onset markers
i1	the index of the first onset coinciding with the beginning of a new bar
offset	the minimum number of onsets in the smallest analysis window
numversions	how many increments of analysis windows
first_bpm	an initial first bpm estimate (optional, can be 0)
prevbpm	the bpm of the previous bar
bpmratio	the ratio between the bpm of the current and the previous bar
accum_time	to calculate the quantized onset markers from durations
cat_pattern	name of the internal array of rhythmic patterns (database)
barnumber	counting the quantized bars
anfile	file pointer of the analysis output
newonsets	file pointer of the quantized onsets
lenres	how many results are being returned by q_process ()
q_results	the list of results returned by q_process ()
resid	index of the selected result in q_results

Table 2: List of command-line arguments

index	variable name	information
1	filename	the path to the file containing the list of note onsets
2	threshold	in seconds, for example 0.1
3	i1	the index of the first onset coinciding with the beginning of the first bar
4	first_bpm	an initial first bpm estimate (optional, can be 0)
5	ternary	flag to set binary or ternary meter
6	offset	the minimum number of onsets in the smallest analysis window
7	numversions	how many increments of analysis windows
8	cat_pattern	name of the internal array of rhythmic patterns

3 Pattern class

Pattern
set_patterns (p) find_pattern (s, strict) find_pattern_strict (s) remove_dups (reslist) beat_align_test (d) flatten_list (input_list)
bpatterns pat_catalog

Table 3: Pattern class methods

symbol	meaning
set_patterns (p)	sets the list of rhythmic patterns to the array named p
find_pattern (s, strict)	checks if the rhythmic pattern s is contained in the current list
find_pattern_strict (s)	checks if the pattern s is exactly equal to a pattern in the current list
remove_dups (reslist)	removes all duplicates from a list
beat_align_test (d)	checks if any of the onsets aligns with 1/3, 1/2, or 2/3
flatten_list (input_list)	flattens a list of lists

Table 4: Pattern class variables

symbol	meaning
bpatterns	the selected list of rhythmic patterns
pat_catalog	the hash table for selecting a list of patterns

4 Cost class

Cost
digest (x) cost_fun (a) cost_fun_euclid (a, b) cost_fun_harm (a) cost_fun_numerator (a) cost_fun_ratios (a) cost_fullbar (a) float_to_ratio (r)

Table 5: Cost class methods

symbol	meaning
digest (x)	The divisibility of the denominator of an integer ratio
cost_fun (a)	The accumulated divisibility cost of all onsets represented as ratios
cost_fun_euclid (a, b)	Euclidean distance between a and b
cost_fun_harm (a)	The rhythmic impact measure
cost_fun_numerator (a)	Like cost_fun but evaluating the numerators instead of the denominators
cost_fun_ratios (a)	Evaluating the ratios of durations in pairs
cost_fullbar (a)	returns the sum of all durations
float_to_ratio (r)	converts a float to an integer ratio in smallest terms

5 Quantizer class

Quantizer
q_process (i1, ternary, offset, numversions) quantize (markers) expwin (n) list_combinations (allsets) downbeat_time (onsetlist, index)

Table 6: Quantizer class methods

symbol	meaning
q_process (i1, ternary, offset, numversions)	spawns numversion quantize processes
quantize (markers)	the actual quantize process
expwin (n)	a partial gaussian window
list_combinations (allsets)	creates all combinations of duration class candidates
downbeat_time (onsetlist, index)	returns an onset time based on index

5.1 quantize method

This method receives a list of onsets created by q_process, which is a small portion only of the list of onsets that was passed to the main program. It first creates a list of durations from that list of onsets. It sorts and groups them according to the conditions that only allows durations to be grouped together if their mutual differences is below a given threshold. This threshold is set by the user when calling main program. The durations

that fall into the same group are called a duration class. Each duration class gets their values normalized and the arithmetic mean for each class is calculated. The quantize method then assumes a list of durations to be expected to happen in the performance of the piece of music. Those durations are modelled as integer ratios p/q that are part of a Farey Sequence F_n . A Farey Sequence represents a list of integer ratios in ascending order and in their lowest terms within the interval $[0...1]$. Usually, not all elements of a Farey Sequence occur in a bar, instead one normally has to do with filtered Farey Sequences, which means that the list of durations is in most cases only a subset of a particular Farey Sequence. The main program also provides an expected number of fundamental beats per bar, i.e. three beats in case of a ternary meter, such as $3/4$, $6/4$, or $9/8$. Or, four beats in case of a binary meter, such as $2/4$, $4/4$, $12/8$. Given a list of ratios from a filtered Farey Sequence, the quantize method calculates some particular values for each of the ratios and stores them as elements of a data structure seen in table 7.

Table 7: The data structure for all expected durations p/q within a bar

index	information
0	the ratio p/q as a float
1	the divisibility cost of q
2	p
3	q

Next, a quasi-Gaussian window w is calculated and stored in a list. The maximum number of quantization candidates per duration class is set (`num_qdurs`). The window w provides an additional weight in the search for a valid quantized duration.

5.1.1 quantization of duration classes

For each duration class, an area of search in the interval $[0...1]$ is set by minimum and maximum values that depend on the arithmetic mean of the duration class. Then, a list of candidates is formed by using this data structure as elements, see table 8.

Table 8: The duration class quantization candidates are stored like this

index	information
0	the ratio p/q as a float
1	the value of w at that point divided by the divisibility cost of q
2	p
3	q
4	w

The list of quantization candidates is sorted after the value of w at the point p/q divided by the divisibility cost of q , in reverse order.

In the next step a list of possible solutions is created that is based on all possible combinations of duration class candidates. The more candidates per duration class are allowed, the longer this list becomes. Hence the need to restrict the number with the variable `num_qdurs`. In an additional step each duration class candidate is placed in the correct order according to the sequence of events manifest in the original onset list. A check is performed to make sure that only those solutions are stored that represent one full bar. The quantization finishes with a list of normalized durations.

5.1.2 results returned by `quantize()`

Finally, the `quantize` method returns a list of data to the `quantize_process` method. The data has the following structure for each item of the list, see table 9.

Table 9: The data structure that captures quantization results

index	information
0	quantized durations (normalized)
1	<code>cost_fun()</code> of quantized durations (small value means low cost)
2	ratio of data at index: 1 divided by length of list at index: 0
3	<code>cost_fullbar()</code> of quantized durations
4	euclidean distance of quantized durations relative to original durations (normalized)
5	rhythmic impact measure, <code>cost_fun_harm()</code> of quantized durations
6	<code>cost_fun_numerator</code> of quantized durations
7	index of onset that starts a new bar
8	tempo of the bar in BPM (average local tempo)

5.2 `q_process` method

This method manages the process of quantization. It delegates all calls to the `quantize` method to multiple processors. Each call receives a different number of onsets to quantize. For each of the `q_process` calls the onset lists starts with the same onset but has a different length. The lengths are being controlled by the parameters `offset` and `numversions`. Each `quantize` call returns a data structure described in table 9, which is collected into a big list, and sorted after the data at index: 2. This is the ratio of the `cost_fun()` of quantized durations divided by the length of the list of quantized durations.

After that, a list of weighted results is created using the structure in table 10 for each element.

Then this list is sorted after the data at index: 1, which is the `beat_align_test` of quantized durations.

Finally, the quantized durations are transcribed into shorthand notation (`snmr`) with `transcribe()`. Then the transcription is compared with a database of rhythmic patterns. The matching results are collected into a data structure and stored in a list named `score_results`, see table 11. The transcribed durations are converted into a simple text string with `snmr_string()`. See also table 13.

Table 10: The data structure of the elements in the list `weighted_results` created by `q_process()`

index	information
0	quantized durations
1	beat_align_test of quantized durations
2	index of onset starting the next bar
3	euclidean distance of quantized durations relative to original durations
4	tempo of the bar in BPM (average local tempo)

Table 11: The `score_results` list data structure of elements

index	information
0	snmr string of transcribed durations
1	beat_align_test of quantized durations
2	index of onset starting the next bar
3	onset time in seconds of the next bar
4	the matching pattern found in the database
5	euclidean distance of quantized durations relative to original durations
6	tempo of the bar in BPM (average local tempo)

`q_process` returns how many results have been found and the list `score_results` to the main program.

5.3 How `main()` handles the return values from `q_process`

The main program calls `q_process` in a loop. After `q_process` returns, main appends the following items to each `score_results` it has received, see table 12.

Table 12: Main appends to the `score_results` elements

index	information
7	ratio between the current and previous BPM
8	previous BPM
9	fitness
10	change in BPM

With the change in BPM being defined as:

$$|1 - b|,$$

where b is the ratio between the BPM of the current bar and the BPM of the previous

bar, the fitness cost g is calculated as follows:

$$g = \frac{d \times |1 - b|}{L}$$

where d is the euclidean distance of the solutions versus the originally performed durations, and L is the length of the list that contains the detected rhythmic pattern.

5.4 Final rules for picking the results

The main program continues with two sorting processes for the result list that is now called `q_results`: The first sort is after the euclidean distance (index 5 of the data structure), the second sort is after the fitness parameter (index 9).

The item that is now at the top of the list is the selected result. But, if the second item of the list has a fitness value within 30% of the top item, then the item that has more duration classes wins. To determine the number of duration classes, the data at index 4 is used, which is the matching pattern that was found in the database.

However, if there are more than five results in `q_results`, and the current bar is not the first one, then `q_results` is sorted after the change in BPM (index 10). And, the first item is picked as the final result.

Main then continues to write the final result to disk using the file pointers `anafile` and `newonsets`. Then, `il` is updated with the data element at index 2, `prevbpm` is updated with the data element at index 6, and the `barnumber` counter is incremented. The loop then continues or ends.

6 Transcription class

Transcription
transcribe (a, tern) transcribe_back (t, tern) transcribe_to_ratios (t, tern) snmr_string (a)
transcript transcript4

Table 13: Transcription class methods

symbol	meaning
transcribe (a, tern)	transcribes a list of durations to shorthand notation
transcribe_back (t, tern)	transcribes a list of shorthand symbols to a list of durations
transcribe_to_ratios (t, tern)	transcribes a list of shorthand symbols to a list of ratios p, q, ...
snmr_string (a)	creates a string of shorthand symbols from a list

Table 14: Transcription class variables

symbol	meaning
transcript	list of normalized duration data in a ternary meter
transcript4	list of normalized duration data in a binary meter